

第四章 字符串

张史梁

课程内容

4.1 字符串的基本概念

4.2 字符串的存储结构实现

4.3 字符串的模式匹配

4.1 基本概念

- 字符串
  - 由0或多个字符顺序排列所组成的有限序列，称“串”
  - 串长度：字符串所包含的字符个数
    - 空串：长度为零的串“”；空格串：“ ”
- 字符串常数：一般用一对双引号 “” 括起来
  - 如“SUNDAY”，“123”，“字符串”等

- 字符(char)：组成字符串的基本单位，取值依赖于字符集Σ（同线性表，结点的有限集合）
  - 二进制字符集：Σ = {0,1}
  - 生物信息中DNA字符集：Σ = {A,C,G,T}
  - 英语语言：Σ = {26个字符，标点符号}
- 字符编码
  - 字符类型是单字节（8 bits）类型
  - 采用ASCII码对128个符号（字符集charset）进行编码 (https://www.asciicode.com/)
  - 国标编码GB2312 （两个字节）、通用文字符号编码标准UNICODE

字符型变量的赋值

```
int main( ) {
    char b, c, d;
    b = 'x' ;
    c = b;
    d = 120;
    return 0;
}
```

b = c =d = 'x'

ASCII 字符代码表 一

高四位 / 低四位		ASCII非打印控制字符										ASCII 打印字符												
		0000					0001					0010		0011		0100		0101		0110		0111		
		0					1					2		3		4		5		6		7		
		十进制	字符	ctrl	代码	字符解释	十进制	字符	ctrl	代码	字符解释	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	ctrl
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p	
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
0010	2	2	☻	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
0011	3	3	♥	^C	ETX	正文结束	19	‼	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	$	52	4	68	D	84	T	100	d	116	t	
0101	5	5	♣	^E	ENQ	查询	21	§	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u	
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
0111	7	7	•	^G	BEL	震铃	23	↨	^W	ETB	传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
1000	8	8	◘	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x	
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y	
1010	A	10	◙	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z	
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{	
1100	C	12	♀	^L	FF	换页/新页	28	∟	^\	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124	|	
1101	D	13	♪	^M	CR	回车	29	↔	^]	GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	♫	^N	SO	移出	30	▲	^6	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ	^Back space

注：表中的ASCII字符可以用:ALT + "小键盘上的数字键" 输入

“怎么比较字符串大小？”

- 为了字符串间比较和运算的便利，字符编码表一般遵循“偏序编码规则”。
- 字符偏序：根据字符的自然含义，某些字符间两两可以比较次序。
  - 其实大多数情况下就是字典序
  - 中文字符串有些特例，例如“笔划”序

encode(‘0’)+1=encode(‘1’)

“monday”<“sunday”

注：encode()是符号的ASCII编码(序号)的映射函数

子串

- 假设s_1，s_2 是两个串
  - $s_1 = a_0a_1a_2...a_{n-1}$和$s_2 = b_0b_1b_2...b_{m-1}$，$(0 \le m \le n)$
  - 若存在整数i $(0 \le i \le n-m)$，使得$b_j = a_{i+j}$，$j = 0,1,...,m-1$

同时成立，则称串s_2是串s_1的子串，或称s_1包含串s_2

- 真子串：非空且不为自身的子串（空串是任意串的子串）
- 任意串S 都是S本身的子串

- 子串处理函数
  - 提取子串
  - 插入子串
  - 删除子串
  - …

思考题

- 若字符串s= “software”，则其子串（真子串）的数目为多少?

答案：37 (35)

抽象数据类型

- 字符串的复杂性源于其长度的动态变化
- 标准字符串(C语言)
  - 采用char S[M]的形式
  - 长度固定
  - 不是面向对象数据类型，存在局限性

  C的字符数组标准库 <string.h>
- 字符串类（class String）（C++语言）
  - 而采用一种动态变长的存储结构
    #include <string>

  (具体的请参考《C++ Primer》的第3.4节)

抽象数据类型(续)

```
class string  {
        private:                        // 字符串的存储结构在具体实现时定义
            char *str                   // 字符串的数据表示
            int size                    // 串的当前长度
        public:                         // 字符串的运算集
            string(char *s = "");       // 创建一个空字符串
            string(char *s);            // 创建一个初值为s的字符串
            ~string ();                 // 消除该串实例
            int length();               // 返回串的长度
            int isEmpty();              // 判断串是否为空串
```

```
    void clear();                                   // 把串清空
    string append(char c);                          // 在串尾添加字符
    string concatenate(char *s);                    // 把串s连接在本串后面
    string copy(char*s);                            // 将一个串s拷贝到本串
    string insert(char c, int index);               // 往串中给定位置插字符
     // 从位置start开始搜索串寻找一个给定字符
    int find(char c, int start);
    // 从位置s开始提取一个长度为len的子串
    string substr(int s, int len);
}
```



4.2 字符串的存储结构与实现

- 静态存储
  - 顺序存储
  - C++标准串的运算实现 （即C风格的字符串）
- 动态存储
  - String类的运算实现

顺序存储：C++标准字符串

- 标准字符串：将C++的<string.h>函数库作为字符串数据类型的方案
  - 采用char S[M]的形式定义字符串变量
  - M是常量，保持不变
- 串的结束标记：'\0'
  - '\0'是ASCII码中8位BIT全0码，又称为NULL
  - 长度为M的空间，实际能够容纳串长为(M-1)

char s1[7]="value.";

V	a	l	u	e	.	\0

char s2[9];

\0								

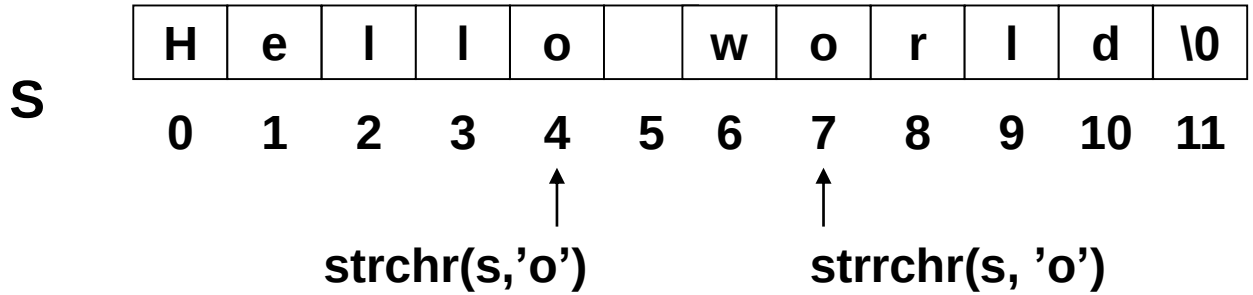
s1,s2实际上就是指向字符串首地址的指针， 拷贝赋值需要一位一位的来做：s1[i]=s2[i]

标准字符串函数

- 串长函数　　//返回字符串s的长度；
  int strlen(char *s);
- 串复制　　//将s2值复制给s1，返回指针指向s1；
  char *strcpy(char *s1, char *s2);
- 串拼接　　//将串s2拼接到s1的尾部
  char *strcat(char *s1, char *s2);
- 串比较
  int strcmp(char *s1, char *s2);（=，>，<）
- (左)定位函数　　//c在s中第一次出现的位置；
  char *strchr(char *s, char c);
- 右定位函数　　//逆向寻找c在s中第一次出现的位置；
  char *strrchr(char *s, char c);

举例

e.g, 字符串s：



寻找字符o, strchr(s,’o’)结果返回4；

反方向寻找o, strrchr(s,’o’)结果返回7

字符串类的存储结构

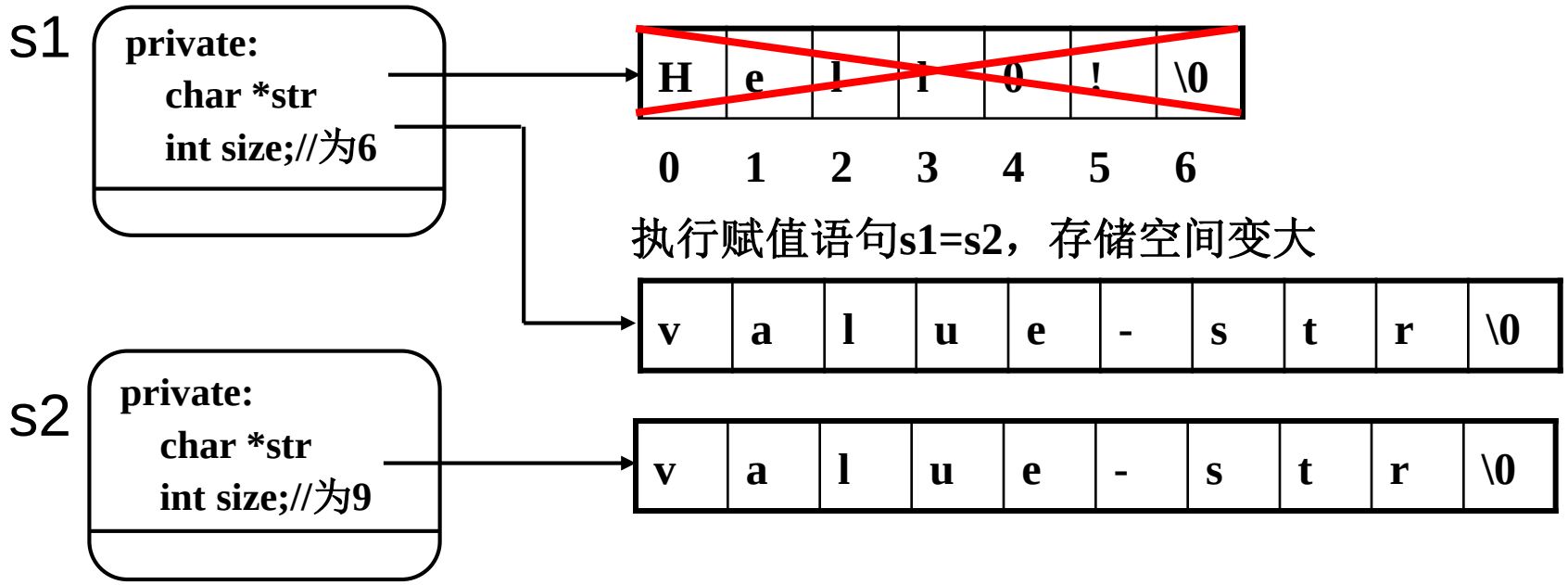
- 字符串类（class String）
  - 适应字符串长度动态变化的复杂性
  - 不再以字符数组char S[M]的形式出现，而采用一种动态变长的存储结构
  - 不适合采用动态链表的形式

C++ String类部分算子列表

操作类别	方法	描述
子串	substr ()	返回一个串的子串
拷贝/交换	swap ()	交换两个串的内容
	copy ()	将一个串拷贝到另一个串中
赋值	assign ()	把一个串、一个字符、一个子串赋值给另一个串
	=	把一个串或一个字符赋值给另一个串中
插入/追加	insert()	在给定位置插入一个字符、多个字符或串
	+=	将一个字符或串追加到另一个串后
	append ()	将一个或多个字符、或串追加在另一个串后
拼接	+	通过将一个串放置在另一个串后面来构建新串
查询	find ()	找到并返回一个子序列的开始位置
替换/清除	replace ()	替换一个指定字符或一个串的字串
	clear ()	清除串中的所有字符
统计	size ()	返回串中字符的数目
	length ()	返回size ()
	max_size ()	返回串允许的最大长度

构造与赋值算子

- 构造函数String::String(char *s)
  - String s1= “hello!”
- 赋值算子String String::operator=(String& s)
  - String s2= “value-str”；s1=s2





字符串运算的算法实现

- C++标准串运算的实现
- String串运算的实现

C++标准串运算的实现

```
int strcmp( char *d, char *s) {   //字符串的比较
    int i =0;
    while (s[i] != '\0' && d[i] != '\0' )   {
        if (d[i] > s[i])
            return 1;
        else if (d[i] < s[i])
            return  -1;
        i ++;
     }
        if( d[i] = ='\0' && s[i] != '\0')
            return -1;
        else if (s[i] = = '\0'&& d[i] != '\0')
            return  1;
        return  0;
}
```

- 寻找字符

```
char * strchr(char *d , char ch) //查找ch在d中第一次出现的位置
{
    i = 0;
    while (d[i] !=  '\0'  && d[i] != ch )  i++;  //跳过不是ch的字符
   if (d[i] = =  '\0' )
        return  '\0' ;                                  //表示失败
    else
        return  &d[i] ;
}
```

- 反向寻找字符

```
char * strrchr(char *d , char ch)
{
    i = 0;
     while (d[i] != '\0' )  i++;                    //找串尾
    while (i >= 0 && d[i] != ch ) i- - ;        //跳过不是ch的字符
    if (i < 0)
            return '\0' ;
    else
            return  &d[i] ;
}
```

String串运算的实现

- 创建算子(constructor)

```
String::String(char *s)
{   //先要确定新创建字符串实际需要的存储空间，s的类型为(char *)，作为新创字符串的初值。确定s的长度，用标准字符串函数strlen(s)计算长度
    size = strlen(s) ;
    //然后在动态存储区开辟一块空间存储初值s，把结束字符也包括进来;
    str = new char [size + 1];
    //开辟空间不成功时，运行异常，退出
    assert(str != NULL);
    //用标准字符串函数strcpy，将s完全复制到指针str所指的存储空间
    strcpy( str, s );
}
```

拼接算子

```
String  String::operator+ (String&  s)
{   //把字符串s和本实例拼接成为一个长串返回
    String temp;                    //创建一个串temp
    int len;
    len = size + s.size;                    //准备工作，计算拼接后的长串的长度
    temp.str= new char [len+1];    //准备工作，开辟空间，为存放长串之用
    assert(temp.str!= NULL);                //若开辟动态存储空间不成功，则退出
    temp.size= len;
    strcpy(temp.str, str);                  //字符串str（以’\0’结尾）存到temp
    strcat(temp.str, s.str);                //再把参数s的str和本实例的str拼接为长串
    return temp;
}
```

赋值算子

```
String  String::operator= (String&  s)
{   //参数s将被赋值到本串,若本串的串长和s的串长不同，则应该释放本串的str存储空间，并开辟新的空间
        if(size != s.size){
            delete [] str ;  //释放原存储空间
            str = new char [s.size+1];
            assert(str != NULL);  //若开辟动态存储空间失败，则退出正常运行
            size = s.size;
        }
        strcpy(str , s.str );
        return  *this; //返回本实例，作为String类的一个实例
}
```

抽取子串函数

```
String String::Substr(int index , int count )
{   //取出一个子串返回，自下标index开始，长度为count
    int i;
    //本串自下标index开始向右数直到串尾，长度为left
    int  left = size – index ;   //自index到串尾的有效长度;
    String temp;
    char  *p, *q;
    //若下标index值太大，超过本串实际串长，则返回空串
    if(index >= size)     // 注意不是 index >= size - 1
            return temp;
    if(count > left )
    //若count超过自index以右的实际子串长度，则count变小
            count = left;
    temp.str = new char [count+1];
    p = temp.str;
    //p的内容是指针，指向目前暂无内容的字符数组首字符处
    q = &str[index];
    //q的内容是一指针，指向本实例串的str数组的下标index
   //q取出所指内容后，指针加1，用p所指的字符单元接受拷贝，指针加1
    for (i =0; i < count;  i++)
            *p++  =  *q++;
    *p = ’ \0’ ;     //循环结束后，让temp.str的结尾为’ \0’
    temp.size = count;
    return temp;
}
```

4.3 字符串的模式匹配

- 模式匹配(Pattern Matching)
  - 一个目标对象T（字符串）
  - 一个模板（pattern）P（字符串）
- 任务
  - 用给定的模板P，在目标字符串T中搜索与模板P全相同的一个子串，并返回P 和T匹配的第一个子串的首字符位置
- 举例
  - T = “easdknjeasdk”，P= “asdk”
  - 返回1



意义

- 是计算机科学中最古老、研究最广泛的问题之一
- 有着大量的实际应用
  - 生物信息学、信息检索、拼写检查、数据压缩检测等
- 例如
  - 在信息检索中，一个挑战性的任务是，搜索出由用户自定义的模式对应在文本中的匹配位置，这种模式很可能带有通配符。
  - 在生物信息学中，相似基因序列的比对。
- 大数据的搜索代价不容小觑！

字符串的模式匹配

- 用给定的模式 P，在目标字符串 T 中搜索与模式 P 全同的一个子串，并求出 T 中第一个和 P 全同匹配的子串（简称为”配串”）,返回其首字符位置

$$
\begin{array}{lccccccccccc}
T & t_0 & t_1 & \cdots & t_i & t_{i+1} & t_{i+2} & \cdots & t_{i+m-2} & t_{i+m-1} & \cdots & t_{n-1} \\
 & & & & \| & \| & \| & & & \| & & \| \\
P & & & & p_0 & p_1 & p_2 & \cdots & p_{m-2} & p_{m-1} & &
\end{array}
$$

为使模式 P 与目标 T 匹配，必须满足

$$p_0\ p_1\ p_2 \cdots p_{m-1} = t_i\ t_{i+1}\ t_{i+2} \cdots t_{i+m-1}$$



模式匹配的目标

- 在大文本（诸如，句子、段落，或书本）中定位（查找）特定的模式
  - 很多算法用于解决模式匹配问题
  - 重点介绍朴素(Brute Force)算法、 Knuth-MorriT-Pratt (KMP) 算法

朴素模式匹配算法 （穷举）

设T= $t_0t_1, t_2, \ldots, t_{n-1}$， P = $p_0, p_1, \ldots, p_{m-1}$

i 为 T 中字符的下标，j 为 P 中字符的下标

匹配成功（$p_0 = t_i, p_1 = t_{i+1}, \ldots, p_{m-1} = t_{i+m-1}$）

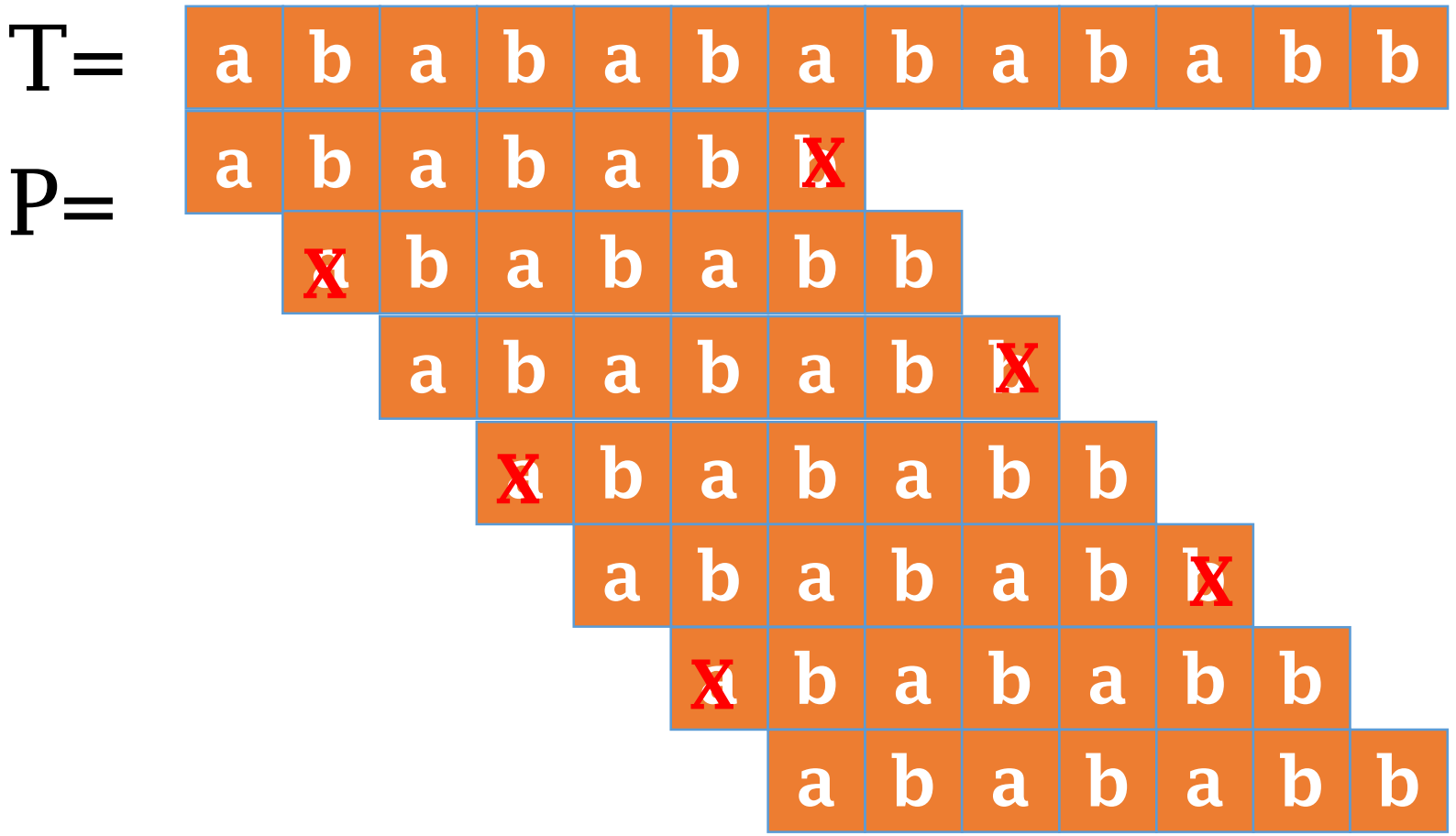
即，T.substr(i, m) == P.substr(0, m)

匹配失败（$p_j \neq t_i$）时，

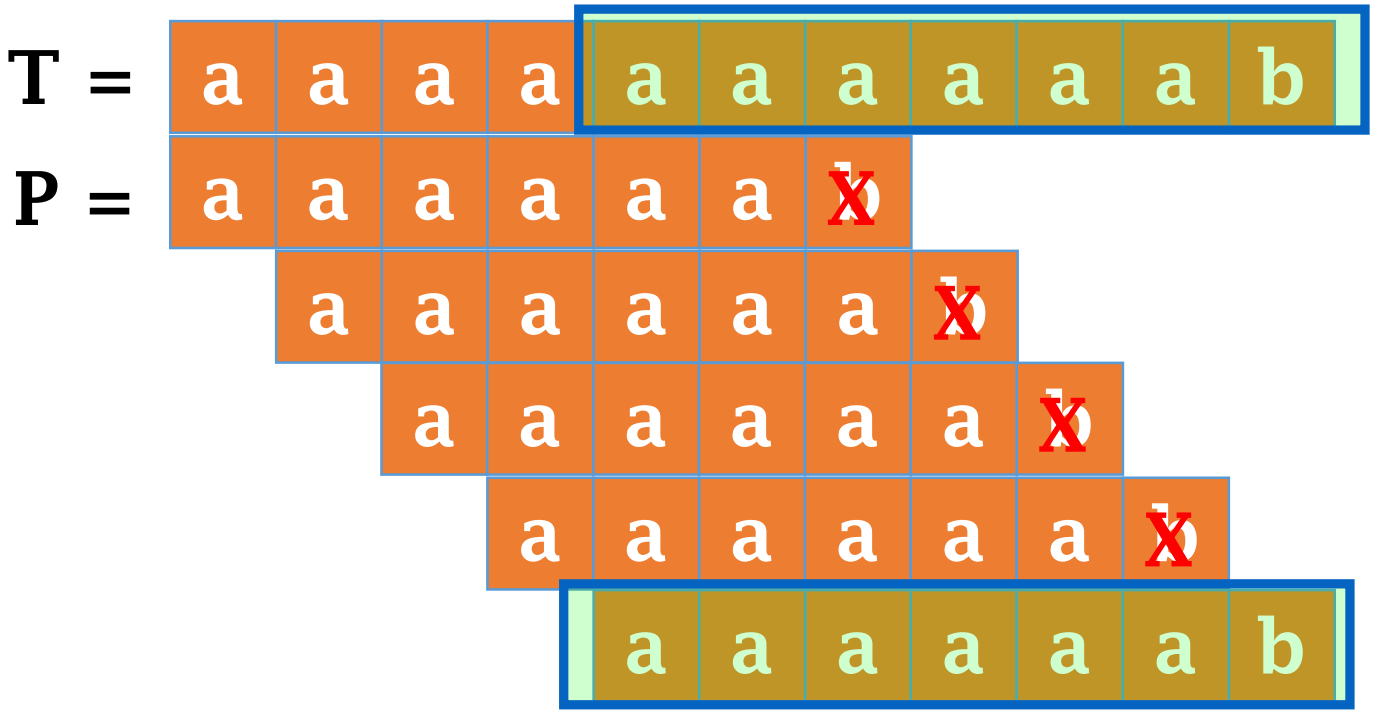
将 P 右移再行比较

尝试所有的可能情况

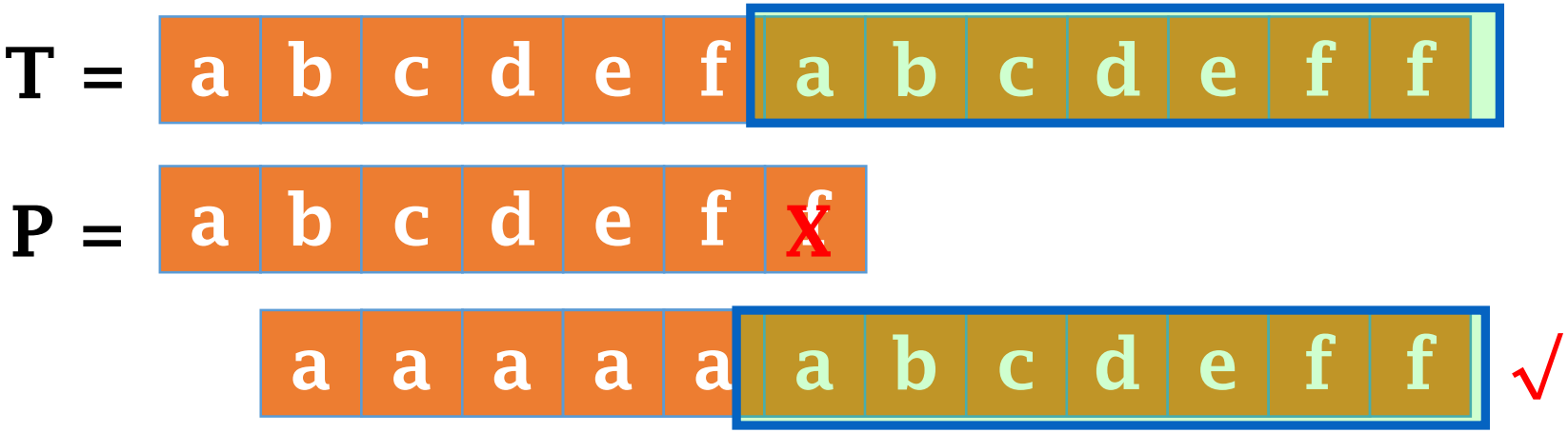
朴素模式匹配例1


T=
a b a b a b a b a b a b b
P=
a b a b a b b
X
a b a b a b b
X
a b a b a b b
X
a b a b a b b
X
a b a b a b b
X
a b a b a b b
X
a b a b a b b

朴素匹配例2


T =
a a a a a a a a a a b
P =
a a a a a a b
X
a a a a a a b
X
a a a a a a b
X
a a a a a a b
X
a a a a a a b

朴素匹配例3



朴素匹配算法1

```
int FindPat_1(string T, string P, int startindex) {
   int LastIndex = T.length() – P.length(); //从S末尾倒数一个模式长度位
置
   int count = P.length();
   if (startindex > LastIndex) //开始匹配位置startindex值过大，无法匹配
         return (-1);
   //g为S的游标，用模式P和S第g位置子串比较，若失败则继续循环
   for (int g = startindex; g <= LastIndex; g++)  {
   if ( P == T.substr(g, count))
          return  g;
  }
  return  (-1);
}
```

朴素匹配算法2

```
int  FindPat_2(string T, string P,int  startindex) {
   int LastIndex = T.length() - P.length();      //从T末尾倒数一个模板长度位置
   if  (startindex > LastIndex) return (-1);     //开始匹配位置值过大，无法匹配
   int i = startindex,   j = 0;                  // i 是T内部游标，j 是P内部游标
   while (i < T.length()  &&  j < P.length())
      if (P[j] == T[i])    {  i++;    j++;   }
      else    {  i = i - j + 1;   j = 0; }
   if ( j  >=  P.length())
      return (i - j);            //若匹配成功，则返回该T子串的开始位置；
   else    return -1;            //若失败，函数返回值为负
}
```

朴素匹配算法3

```
int FindPat_3(string T, string P, int startindex) {
   //g为T的游标，j为P的游标
   for (int g= startindex; g <= T.length() - P.length(); g++) {
      for (int j=0;  ((j<P.length()) && (T[g+j]==P[j])) ; j++)
         ;
      if (j == P.length())
         return  g;
   }
   return(-1);  // for结束，或startindex值过大,则匹配失败
}
```



朴素匹配算法：最佳情况

- 在目标的前M个位置上找到模式，设 M = 5

AAAAAAAAAAAAAAAAAAAAH

AAAAA　　5次比较

- **总比较次数：M**
- **时间复杂度：O(M)**

朴素匹配算法：最差情况

- 模式与目标的每一个长度为M的子串进行比较

  - 目标形如a^n, 模式形如$a^{m-1}b$
  - 总比较次数：
    - M(N-M+1)
  - 时间复杂度：
    - O(M*N)

```
AAAAAAAAAAAAAAAAAAAAAH
AAAAH          5次比较
AAAAAAAAAAAAAAAAAAAAAH
 AAAAH         5次比较
AAAAAAAAAAAAAAAAAAAAAH
  AAAAH        5次比较
AAAAAAAAAAAAAAAAAAAAAH
   AAAAH       5次比较
AAAAAAAAAAAAAAAAAAAAAH
    AAAAH      5次比较
.....
AAAAAAAAAAAAAAAAAAAAAH
                AAAAH
```

朴素算法性能分析

- 分析
  - 假定目标T的长度为n，模板P长度为m， $m \leq n$
  - 在最坏的情况下，每一次循环都不成功，则串p一共右移（$n-m+1$）次
  - 每一次“相同匹配”比较所耗费的时间，是P和T逐个字符比较的时间，最坏情况下，共m次。
  - 因此，整个算法的最坏时间开销估计为$O(m * n)$。

有优化的算法吗?



一起观察分析下面的例子

T: a b a c a a b a c c a b a c a b a a
P: a b a c a b

1）$P_5 \neq T_5$ P右移一位

T: a b a c a a b a c c a b a c a b a a
P: a b a c a b

2）$P_0 \neq T_1$ P右移一位

T: a b a c a a b a c c a b a c a b a a
P: a b a c a b

3）$P_1 \neq T_3$ P右移一位

T: a b a c a a b a c c a b a c a b a a
P: a b a c a b

4）$P_0 \neq T_3$ P右移，得到$P_0 = T_4$

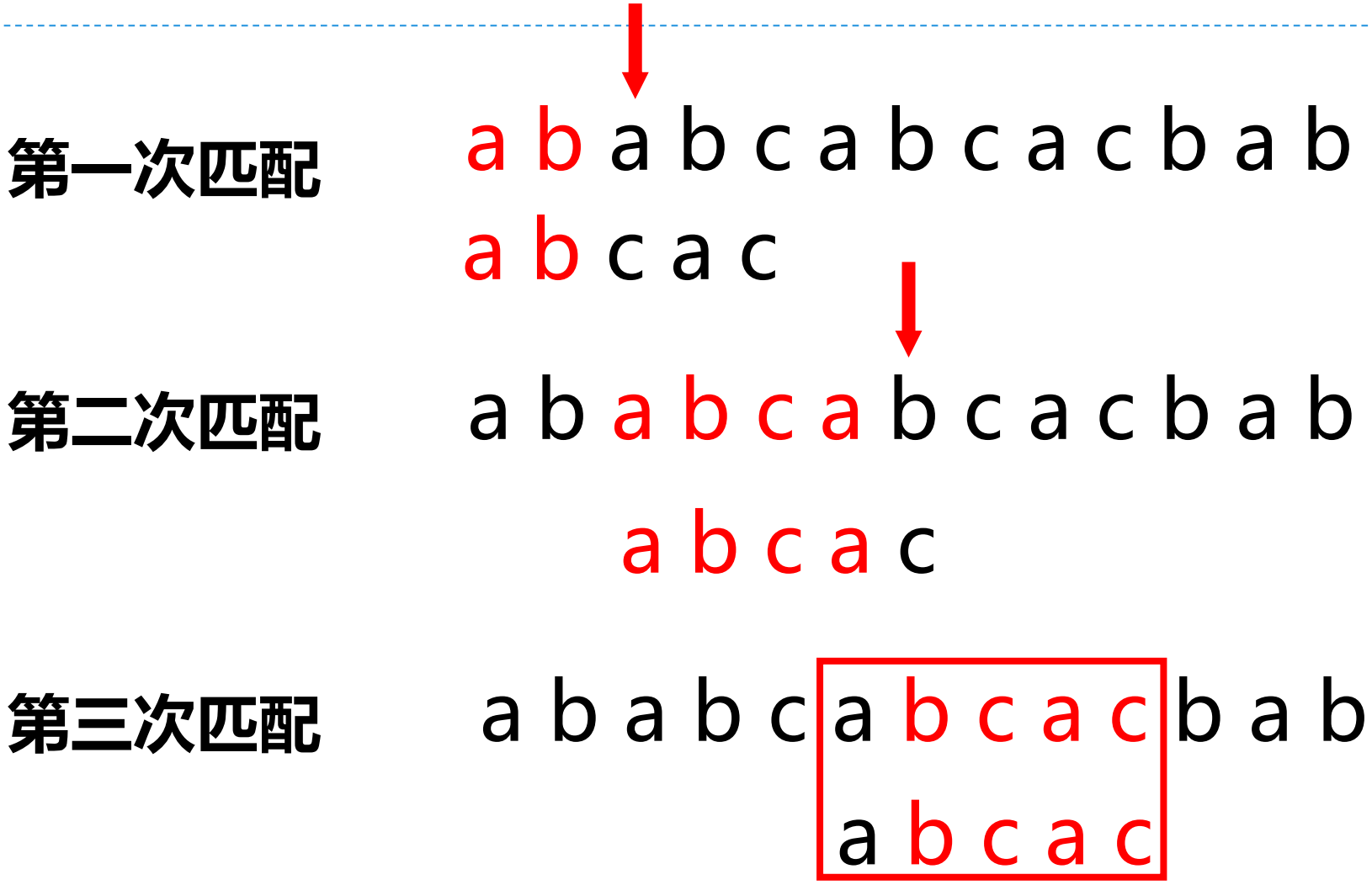
T: a b a c a a b a c c a b a c a b a a
P: a b a c a b

直到 $P_0 = T_4$，开始$P_1 = T_5$

- 由例可知： $P_5 \neq T_5$，
  $P_0P_1P_2P_3P_4=T_0T_1T_2T_3T_4$，同时由$P_0 \neq T_1$可知，
  将P右移一位后第2趟比较一定不等
- 比较操作： $P_0 \neq T_1$、$P_1 \neq T_3$、$P_0 \neq T_3$、$P_0 = T_4$都是可以从$P_0P_1P_2P_3P_4$推断出来的，
- $P_1 = T_5$才是有效比较，其余比较都是冗余

消除冗余比较而不丢失配串

希望能达到如此效果


第一次匹配
a b a b c a b c a c b a b
a b c a c
第二次匹配
a b a b c a b c a c b a b
a b c a c
第三次匹配
a b a b c a b c a c b a b
a b c a c

无回溯匹配

- 寻找一个无回溯(T的游标不后退)的匹配算法，关键在于匹配过程中，一旦 P_i 和T_j比较不等时，即

  P.substr(0, i-1) = T.substr(j-i, j-1)　且　$P_i \neq T_j$

- 要能立即确定右移的位数，即该用P中的哪个字符和T_j进行比较?

如何移位？移多少位?

如何利用匹配失败位置前的信息，消除大量不必要的回溯？



KMP算法

- Knuth-MorriT-Pratt (KMP)算法发现，P中每个字符对应一个移位值k，该值仅依赖于模式P本身，与目标T无关
  - 1970年，S. A. Cook在进行抽象机的理论研究时证明了在最差情况下模式匹配也可在N+M时间内完成
  - 此后，D. E. Knuth 和V. R. Pratt以此理论为基础，构造了一种方法来在N+M时间内进行模式匹配；
  - 与此同时，J. H. Morris在开发文本编辑器时为了避免在检索文本时的回溯也得到了同样的算法。

Donald E. Knuth



Professor Emeritus of The Art of Computer Programming at Stanford University

1938年出生于Wisconsin。1960年，当他毕业于凯斯理工学院(Case Institute of Technology)数学系时，因为成绩过于出色，被校方打破历史惯例，同时授予学士和硕士学位。他随即进入大名鼎鼎的加州理工学院数学系，仅用三年时间便取得博士学位，此时年仅25岁。 毕业后留校任助理教授，28岁时升为副教授。30岁时，加盟斯坦福大学计算机系，任正教授。从31岁那年起，他开始出版他的历史性经典巨著：The Art of Computer Programming。他计划共写7卷，然而仅仅出版三卷之后，已经震惊世界，使他获得计算机科学界的最高荣誉Turing Award!此时，他年仅38岁！后来，此书与牛顿的“自然哲学的数学原理”等一起，被评为“世界历史上最伟大的十种科学著作”之一。

牛人的思路

T $s_0\ s_1 \cdots s_{j-i-1}$ $\boxed{s_{j-i}\ s_{j-i+1}\ s_{j-i+2} \cdots s_{j-2}\ s_{j-1}}$ $\boxed{s_j}$ $\cdots s_{n-1}$

↕ ↕ ↕ … ↕ ↕ ×

P $p_0\ p_1\ p_2 \cdots p_{i-2}\ p_{i-1}\ p_i$

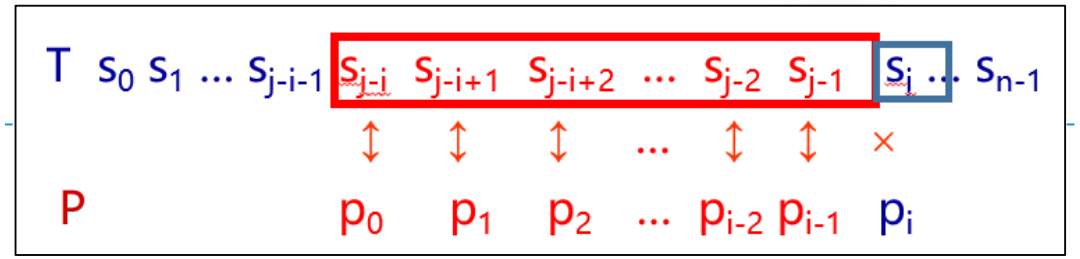
则有 $s_{j-i}\ s_{j-i+1}\ s_{j-i+2} \cdots s_{j-1} = p_0\ p_1\ p_2 \cdots p_{i-1}$

(1) 如果 $p_0\ p_1 \cdots p_{i-2} \neq p_1\ p_2 \cdots p_{i-1}$

(2) 则立刻可以断定

$$p_0\ p_1 \cdots p_{i-2} \neq s_{j-i+1}\ s_{j-i+2} \cdots s_{j-1}$$

(朴素匹配的)下一趟一定不匹配，可以跳过去



同样，若 $p_0\ p_1 \cdots p_{i-3} \neq p_2\ p_3 \cdots p_{i-1}$

则再下一趟也不匹配，因为有

$$p_0\ p_1 \cdots p_{i-3} \neq s_{j-i+2}\ s_{j-i+3} \cdots s_{j-1}$$

直到对于某一个“k”值(首尾串长度)，使得

$$p_0\ p_1 \cdots p_{k-1} = p_{i-k}\ p_{i-k+1} \cdots p_{i-1}$$

则

$$p_0\ p_1 \cdots p_{k-1} = s_{j-k}\ s_{j-k+1} \cdots s_{j-1}\ s_j$$

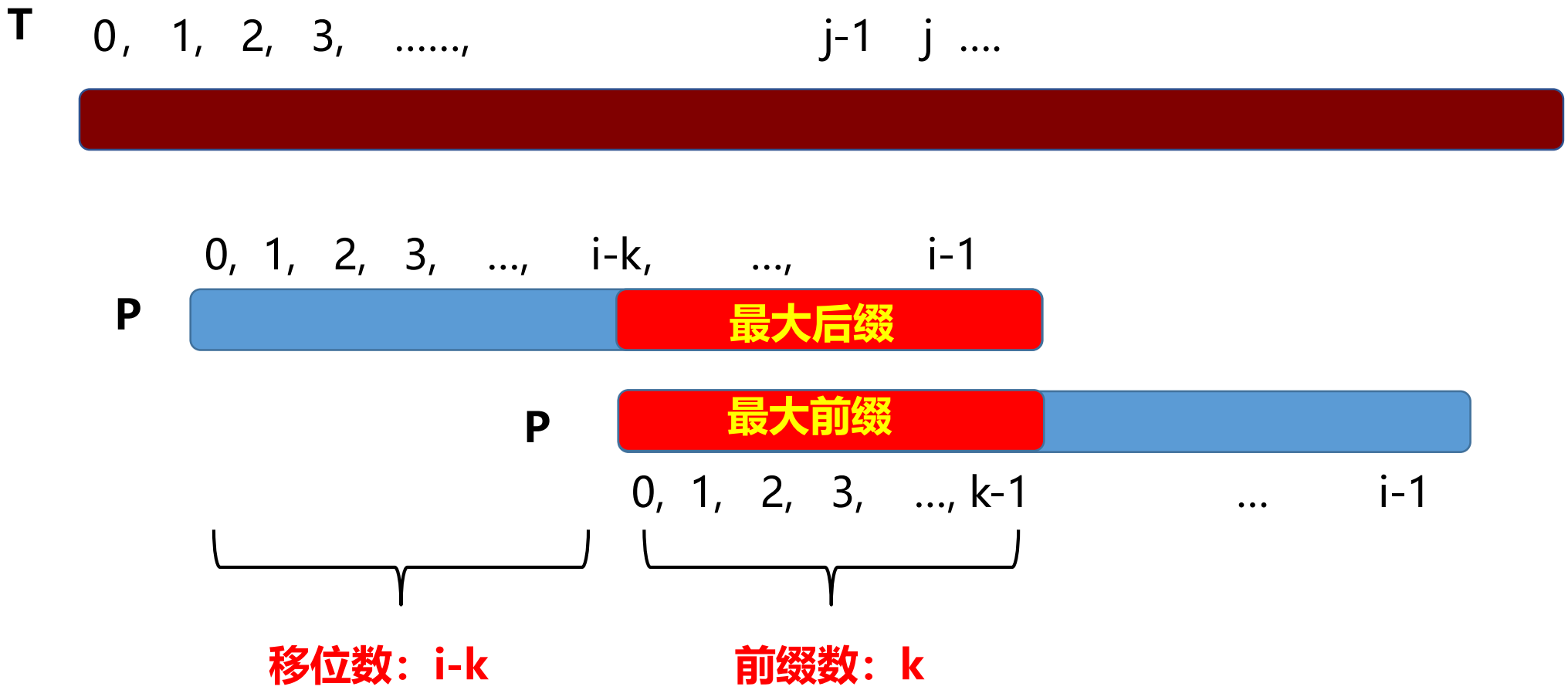
$$\|\quad \|\quad \cdots\quad \|\quad \neq$$

$$p_{i-k}\ p_{i-k+1} \cdots\ p_{i-1}\ p_i$$

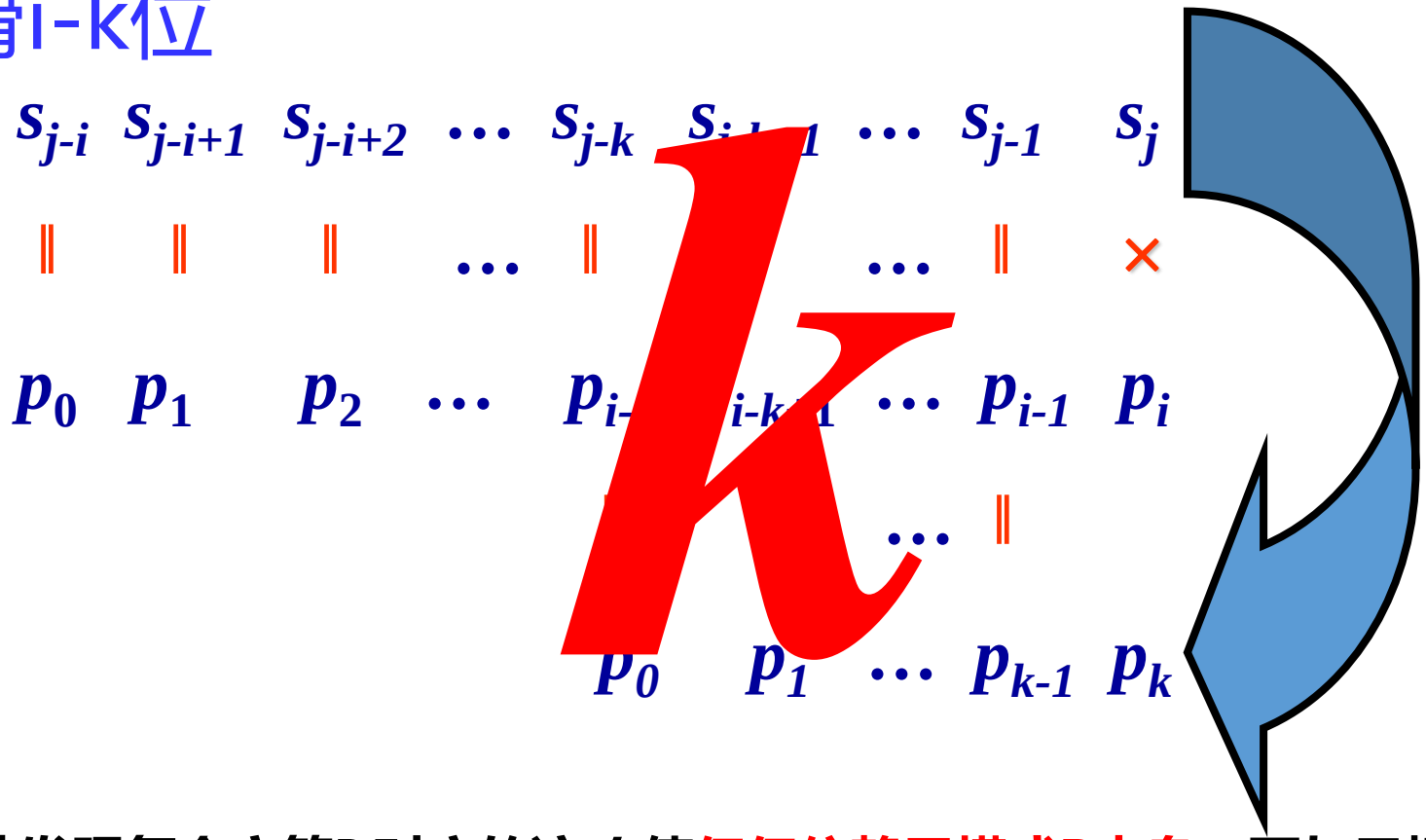
$$\|\quad \|\quad \cdots\quad \|\quad ?$$

$$p_0\quad p_1\quad \cdots\quad p_{k-1}\quad p_k$$

思路：寻找最长相等前后缀



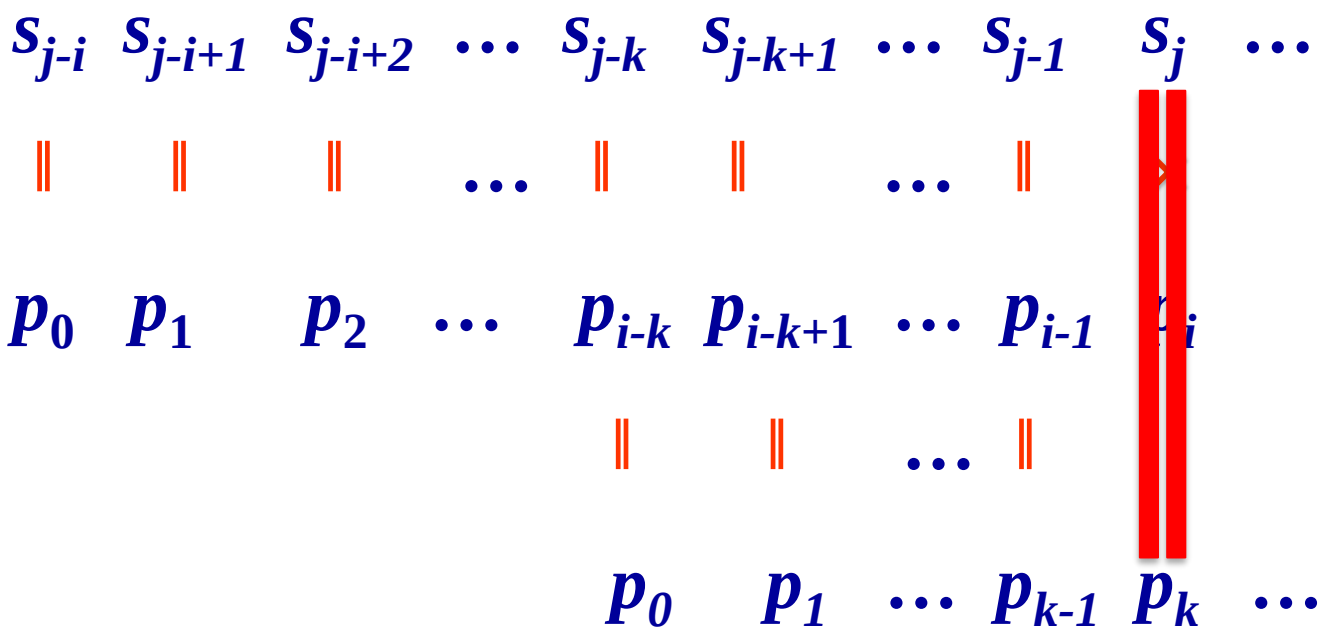
- 模式右滑i-k位



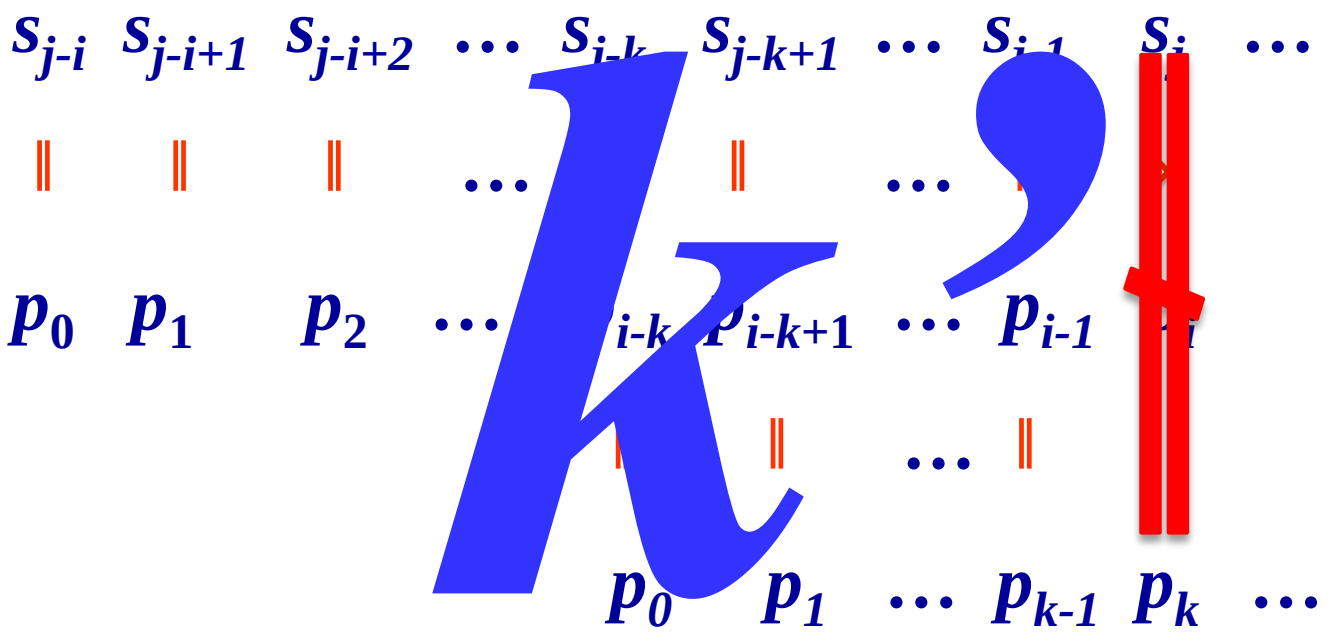
KMP算法发现每个字符P_i对应的该 k 值仅仅依赖于模式P本身，而与目标对象T无关

移位后，继续匹配

- 如果$s_{j==}p_k$，则继续匹配！



- 如果$s_j \neq p_k$，怎么处理?



- k′ 是什么?
- 这个过程会继续下去吗？何时停止?

每个字符对应的支持快速偏移的K值，但是该如何确定和保存?



字符串的特征向量N

- 设模板P由m个字符组成，记为

$$P = p_0 p_1 p_2 p_3 \cdots\cdots p_{m-1}$$

- 令特征向量N 用来表示模板 P 的字符分布特征，简称N向量
  - 和P同长，由m个特征数$n_0 \ldots n_{m-1}$非负整数组成，记为

$$N = n_0\ n_1\ n_2\ n_3\ \ldots\ldots\ n_{m-1}$$

- N在很多文献中也称为next数组，每个n_i对应 next数组中的元素next[i]

计算特征数 n_i

思路：寻找最长相等前后缀

- 前后缀定义
  - 列出模板P开头的任意k个连续字符，把它称为P的前缀子串(k<i)，记做$p_0p_1p_2...p_{k-1}$
  - 在P的第i位置的左边，也取出k个字符，称为P的后缀子串(k<i)，记做$p_{i-k}...p_{i-2}p_{i-1}$
- 设法求出最长的（k最大的）能够与前缀子串匹配的后缀子串
- k就是要求的特征数n_i

结论：n_i 等于$p0...p_{i-1}$中相同并且最大的前缀和后缀的长度($0 \le k < i-1$)。

特征数n_i的递归定义

$$next[i]=\begin{cases}-1, & \text{对于 } i=0\\ \max\{k: 0<k<i \ \&\& \ P(0...k\text{-}1)=P(i\text{-}k...i\text{-}1)\ \}, & \text{如果k存在}\\ 0, & \text{否则}\end{cases}$$

- 特征数n_i ($-1 \le n_i \le i$)是递归定义的，定义如下：
  1. $n_0 \leftarrow -1$，对于$i > 0$的n_i，假定已知前一位置的特征数n_{i-1}，并且$n_{i-1} = k$；
  2. 如果$p_i = p_k$，则$n_{i+1} \leftarrow k+1$；
  3. 当$p_i \neq p_k$且$k \neq 0$时，则令$k \leftarrow n_k$；循环直到条件不满足；
  4. 当$p_i \neq p_k$且$k = 0$时，则$n_{i+1} = 0$;

next数组计算 - 程序实现

- 假定已经计算得到next[i], 那么next[i+1] = ?

  对于next[i]=k，有： $p_0 \cdots p_{k-1} = p_{i-k} \cdots p_{i-1}$

  (1) 如果$p_k=p_i$，则有：$p_0 \cdots p_{k-1}p_k = p_{i-k} \cdots p_{i-1}p_i$

  next[i+1] = k+1 = next[i]+1

  (2) 如果$p_k \neq p_i$，此时需要在$p_0 \cdots p_{k-1}p_k$中寻找最长前缀，使其和 $p_{i-k} \cdots p_{i-1}p_i$的最长后缀相同

  与模式匹配的思路一样，应当将模式串往右滑动到模式串的第next[k]个字符与p_i进行比较，即 k=next[k]。 如果$p_k \neq p_i$则继续滑动。

观察

a b a a b c a c

a b a a b c a c

-1 0 0 1 1 2 0 1

Exercise

- a b c a b a a b c b c

  -1 0 0 0 1 2 1 1 2 3 0

- a b a b a b a b

  -1 0 0 1 2 3 4 5

- a a b a a c a a d a

  -1 0 1 0 1 2 0 1 2 0

特征向量N：非优化算法

```
int findNext(string P) {
    int i, k;
    int m = P.length( );              //m为模板P的长度
    int *next = new  int[m];          //动态存储区开辟整数数组
    next[0] = -1;
    i = 0;  k = -1;
    while (i < m-1) {                 // 若写成 i < m 会越界
       //如果不等，采用 KMP 方法自找首尾子串
       while (k >= 0 && P[k] != P[i])
          k = next[k];                // k 递归地向前找
       i++;   k++;
       next[i] = k;
    }
    return next;
}
```

计算示例

- p="abcdaabcab"

下标i	0	1	2	3	4	5	6	7	8	9
p_i	a	b	c	d	a	a	b	c	a	b
next	-1	0	0	0	0	1	1	2	3	1

如果$P_i=P_k$时，上述算法是否存在进一步优化的空间?

特征向量N：优化算法

```
int findNext*(string P) {
    int i, k;
    int m = P.length( );              //m为模板P的长度
    int *next = new  int[m];          //动态存储区开辟整数数组
    next[0] = -1;
    i = 0;  k = -1;
    while (i < m-1) {                 //若写成i < m 会越界
        while (k >= 0 && P[k] != P[i]) //找首尾子串
              k = next[k];            //k递归地向前找
        i++;   k++;
        if (P[k] == P[i])
              next[i] = next[k];      //前面找k值，优化步骤
        else next[i] = k;
    }
    return next;
}
```

计算示例

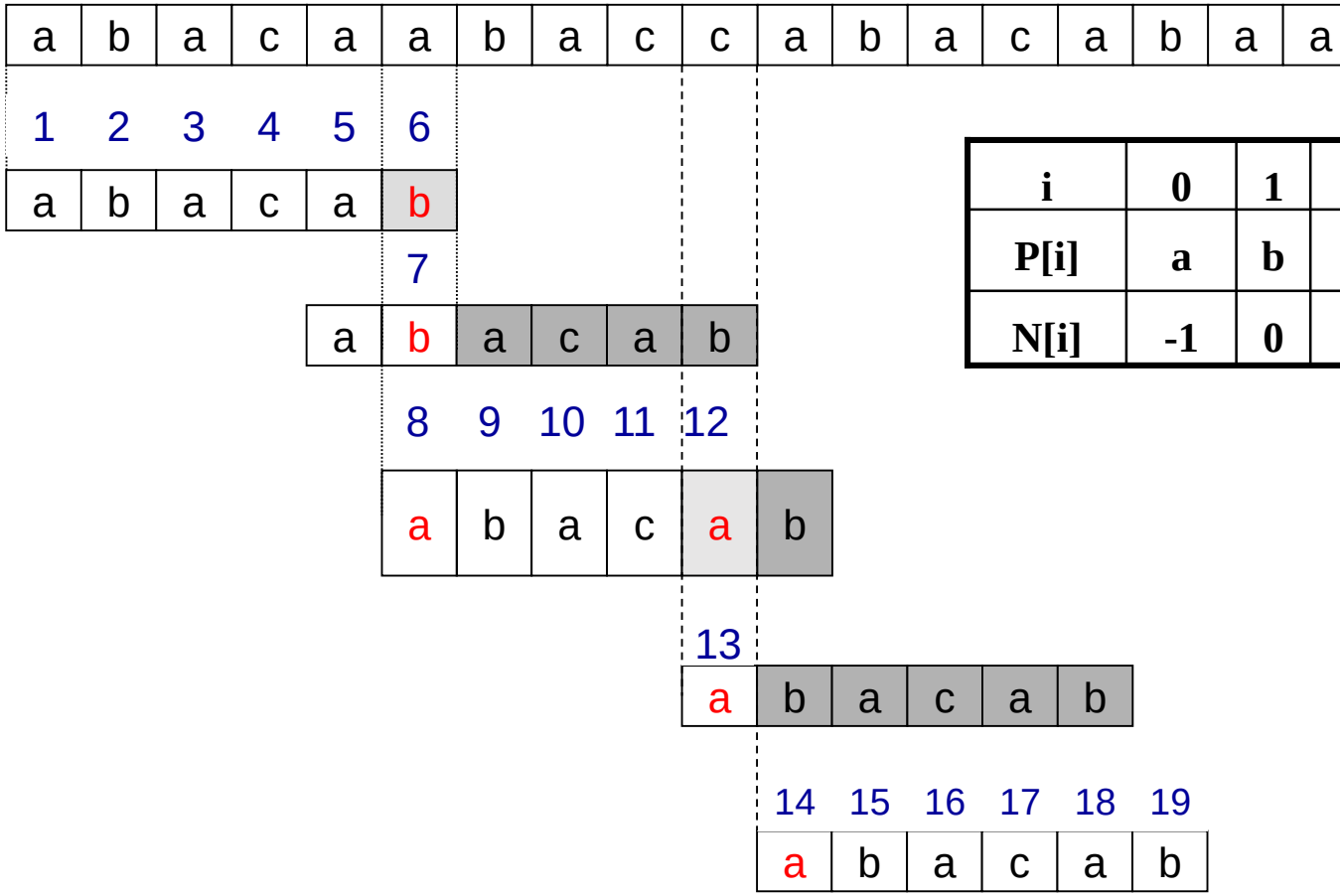
- p="abcdaabcab"

下标i	0	1	2	3	4	5	6	7	8	9
p_i	a	b	c	d	a	a	b	c	a	b
next	-1	0	0	0	0	1	1	2	3	1
next*	-1	0	0	0	-1	1	0	0	3	0

KMP模式匹配算法

```
int KMPStrMatching(string T, string P,  int *N, int start) {
    int i= 0;                                  // 模式的下标变量
    int j = start;                             // 目标的下标变量
    int pLen = P.length( );                    // 模式的长度
    int tLen = T.length( );                    // 目标的长度
    if (tLen - start < pLen)                   // 若目标比模式短，匹配无法成功
        return (-1);
    while ( i < pLen  &&  j < tLen)  {  // 反复比较对应字符来开始匹配
        if ( i == -1  ||  T[j] == P[i])
            i++,  j++;
        else i = Next[i];
    }
    if (i >= pLen)
        return (j-pLen+1);
    else return (-1);
}
```

KMP模式匹配：示例



i	0	1	2	3	4	5
P[i]	a	b	a	c	a	b
N[i]	-1	0	0	1	0	1/0

KMP算法的效率

- KMP的算法的时间复杂性为O(n+m)
  - while循环语句中，j只增不减，所以循环体中的”j++, i++;” 语句执行次数最多n次。
  - “i = N[i];”语句的执行次数也不能超过 n 次。
    - 由于“i = N[i]; ”每执行一次必然使得i至少减1。而使得 i 增加的操作只有“i++” 。那么，如果“i = N[i]; ”的执行次数超过n次，最终的结果必然使得 i 为比-1小很多的负数。这是不可能的（i有时为-1,但是很快+1回到0）。
  - 所以整个循环体的时间复杂度为O(n)
- 同理可以分析出求next数组的时间为O(m)
- 因此，KMP算法的时间为O(n+m)



总结

- 字符串抽象数据类型
- 字符串的存储结构和类定义
- 字符串运算的算法实现
- 字符串的模式匹配
  - 特征向量N及相应的KMP算法还有其他变种、优化

练习

- 设S1, S2为串，请给出使S1+S2=S2+S1成立的所有可能的条件（其中+ 为连接运算）；

  s1或s2至少一个为空；s1==s2；s1、s2分别为一个前缀的若干倍

- 设计一个算法来实现字符串逆序存储，要求不另设串存储空间；

  利用字符串的结束符'\0'，作为中间变量

  利用a=a+b;b=a-b;a=a-b;

  利用a=a^b;b=a^b;a=a^b;（异或）

- 对t=“ababbaaaba”, p = “aab” 进行KMP 快速模式匹配，请画出匹配过程的图示。

练习

```
makeNext( PSeqString p, int *next )
{
    int i = 0, k = -1;   next[0] = -1;
    while ( i < p->n-1 )   /* 计算next[i+1] */
    {
        while (k >= 0 && p->c[i] != p->c[k])
            k = next[k];    /* p0…pi中最大的相同的前后缀长度
             k */
        i++;  k++;
        if (p->c[i] == p->c[k])      next[i] = next[k];
        else       next[i] = k;
    }
}
```

下标i	0	1	2
p_i	a	a	b
最大相同前后缀长度k	-1	0	1
P_k与p_i比较		=	≠
next[i]	-1	-1	1

a	b	a	b	b	a	a	a	b	a
a	a								
		a	a						
				a	a	b			
					a	a	b		
						a	**a**	**b**	



数据结构与算法